

ShopPilot: An Offline-First Conversational Shopping Agent for Constraint-Grounded Multi-Turn Retrieval

TikTok TechJam 2026 — Track 4 (Shopping Copilot) Participant Submission

TikTok TechJam 2026 · Track 4 Shopping Copilot · IEEE-style technical report (hackathon documentation)

Abstract—Conversational e-commerce agents must map ambiguous, multi-turn natural language into a ranked shortlist from a large catalog while minimizing dialogue length. We present ShopPilot, an offline-first shopping agent built on the official TechJam 2026 conversational-search kit (50,000 Clothing_Shoes_and_Jewelry products; 200 public evaluation sessions). ShopPilot combines (i) dual-track buying/browsing intent cues, (ii) a compact dialog state with soft/disclosed/override slot provenance, (iii) in-memory hybrid retrieval (SQLite FTS5 BM25 plus optional hashed character n-gram dense lane), and (iv) constraint-coverage reranking with early-turn precision control and category-tail exact bonuses. On the public set, the default hash path achieves Hit Rate@10 = 0.970, MRR = 0.836, MTTC = 2.93, and TechnicalScore = 0.897 versus the weak BM25 starter (0.125 / 0.068 / 9.81 / 0.107), using zero model tokens. We report per-scenario metrics, ablation-informed design choices, limitations, and reproducibility details.

Index Terms—conversational recommendation, multi-turn retrieval, dialog state tracking, hybrid search, e-commerce agents, offline evaluation.

I. Introduction

Traditional product search optimizes a single query against a static index. Modern shoppers instead refine goals across turns: they open with vague category language, disclose hard constraints when asked, change their mind mid-session, or declare that some attributes do not matter. The TechJam 2026 Track 4 challenge formalizes this setting. Each session hides a target parent_asin in a frozen Amazon Reviews 2023 catalog subset. An agent may take at most ten turns; each turn must return a natural-language message, at most one structured ask_attribute from a fixed enum, and a Top-10 recommendation list. Scoring emphasizes coverage (Hit Rate@10), ranking quality (MRR), and efficiency (mean turns to conversion, MTTC).

The organizer-provided weak BM25 starter is intentionally underpowered (TechnicalScore approximately 0.107 on 200 public sessions). Rebuilding a full LLM stack is unnecessary for a strong entry and can harm feasibility (API keys, latency, non-determinism). ShopPilot therefore targets a different design point: a deterministic, network-optional agent whose default path uses only the Python standard library (plus optional NumPy for a hashed dense lane).

Contributions. (1) A production-shaped multi-turn agent interface fully compatible with the official evaluator. (2) A dialog-state design that separates soft preferences from disclosed facts and implements soft-only wipe on intent override. (3) A hybrid retrieve–rerank stack with precision-first early turns, category-tail bonuses, and an other-first clarification policy matched to the simulator. (4) A fully offline public-set result of TechnicalScore 0.897 with 0 tokens, with scenario breakdowns and documented negative ablations (e.g., pure max information-gain ask order).

II. Related Work and Problem Framing

Conversational recommendation (CRS) and task-oriented dialog systems classically separate natural language understanding (slot filling), dialog management (what to ask next), and retrieval/ranking. ShopPilot follows that pipeline under hackathon constraints—in-memory execution, read-only catalog,

no ASIN injection, and a protocol-legal ask_attribute vocabulary—rather than open-ended LLM dialogue. We organize prior work into four clusters that map onto our modules, then state how the TechJam task instantiates them.

A. Clarifying Questions in Search and CRS

Asking clarifying questions (CQs) is a first-class lever for multi-turn retrieval. Qulac showed that CQ selection should condition on the original query and prior question–answer interactions, and that even one well-chosen question can yield large gains in early precision (e.g., P@1) [1]. In CRS, learning to ask appropriate questions is repeatedly identified as the key to tracing dynamic preferences and reaching accurate recommendations in fewer turns [8]. Zero-shot CQ generation for conversational search further emphasizes templates and query facets to keep questions effective and precise without large labeled CQ corpora [9]. Preference-elicitation work with large language models studies funnel-style questioning—broad concepts first, then finer attributes as the dialog proceeds [4].

ShopPilot instantiates these ideas without a learned CQ generator: every turn pairs a Top-K list with exactly one enum-constrained ask; the first ask is the simulator catch-all other (broad disclosure), then a static facet order (color → material → style → ...) analogous to a shallow funnel [4], [9]. Message wording is optionally grounded in facets observed in the live candidate pool (Section III-D).

B. Information Gain, User Limits, and Corpus Grounding

Bayesian CRS formulations select the question that minimizes conditional entropy (maximizes expected information gain) and may stop when posterior entropy is low [2]. Empirically, users will answer a substantial number of product CQs on average, yet fatigue and irrelevant questions cause early abandonment, and a non-trivial fraction of answers can contradict the eventual target description [3]. Separately, corpus-informed CQ generation argues that questions should be a function of the repository and the information it actually contains; otherwise generators “hallucinate” intents absent from the catalog [5].

These results jointly motivate ShopPilot’s design tensions. MTTC is an explicit efficiency objective under a hard 10-turn cap, consistent with fatigue bounds [3]. Boundary don’t-care handling covers preference refusal. Critically, we evaluated a pure maximum information-gain ask policy over the live candidate pool—the textbook recommendation from [2]—and observed a large TechnicalScore drop on the official public simulator (historical ≈0.72 vs. static+other-first). High-entropy brand/store splits often fail to match the simulator’s hidden constraint classifier, so unconstrained IG is not automatically optimal under protocol-mismatched oracles. We therefore keep a simulator-aligned static order while still using pool-derived facet tags for message grounding, in the spirit of corpus-informed CQs [5] without free-form generation.

C. Dialog State Tracking, Intents, and Slots

Dialogue state tracking (DST) maintains structured slot–value constraints across turns rather than relying on raw history alone [6], [13], [14]. Scalable multi-domain DST represents state over candidate value sets derived from history and knowledge [6]. Joint models map utterances to intents and slots (e.g., profile-conditioned intent–slot models [11] and explicit multi-intent slot mapping [12]). Consumer-type-aware CRS adapts recommendation granularity and attribute-query complexity to user type [10].

ShopPilot’s SessionState is a compact, rule-based DST: ordered constraints with provenance tags soft / disclosed / override, asked and don’t-care sets, internal family/audience labels, and a history cursor that drops pre-override turns from retrieval. Soft-only wipe on intent override is our task-specific

state-update rule: abandoned preferences must not re-enter FTS, while disclosed hard facts (which the simulator will not re-send) are retained. Buying vs. browsing cues play a lightweight role analogous to type- or intent-adaptive policy [10], without a trained consumer-type classifier. Optional LLM slot normalization (off by default) is schema-validated against the official ask enum, closer to constrained slot filling than open generation [11], [12].

D. Challenge Framing and System Position

The TechJam 2026 conversational-search kit defines the catalog, Agent API, scenarios (buying, browsing, intent override, boundary), and metrics [16]. The four pillars map to modules as follows: (I) intent routing and hybrid multi-route retrieval; (II) multi-turn state evolution including override and don't-care; (III) context programming via accumulated slots and query construction; (IV) evaluation on Hit@10, MRR, and MTTC with

$\text{TechnicalScore} = 0.50 \cdot \text{Hit@10} + 0.30 \cdot \text{MRR} + 0.20 \cdot \text{clip}((11 - \text{MTTC})/10, 0, 1)$.

Dense retrievers and cross-encoders can improve paraphrase recall but add dependencies; our default path remains lexical FTS5 plus optional hashed n-gram dense fusion. Positioning. ShopPilot is not a reimplement of Qulac, Bayesian CRS, KBQG, or LLM funnel agents [1], [2], [4], [8]. It is an offline, enum-constrained retrieve-and-ask agent that borrows their problem structure—CQ quality, statefulness, corpus grounding, turn efficiency—and validates design choices with public-set ablations, including a negative result for unconstrained max-IG ask selection on this simulator.

III. System Architecture

ShopPilot implements the kit's Agent API (reset / respond). On each user utterance the agent executes: ingest and slot update → optional query rewrite → hybrid candidate generation → constraint-aware scoring and ranking → ask policy → response assembly. Figure conceptually: User utterance → NLU/ingest (constraints, family, audience, override) → SessionState → FTS5 + dense recall → rerank (coverage, category tail, combo exact, audience/family) → Top-K (precision gate) + one ask_attribute → evaluator.

A. Dialog State and Intent / Override Hygiene

SessionState stores ordered constraints with provenance tags soft, disclosed, or override; parallel sets for asked attributes and don't-care attributes; product family and audience (internal, not official asks); message history with a cursor that drops pre-override turns from retrieval; and flags such as override_applied.

Intent cues. Buying vs browsing is inferred from opening language (targeted requirements vs open exploration). This biases filtering strictness and message tone without requiring a trained classifier.

Override. Patterns such as “ignore my earlier preference,” “change of plans,” or “switch to ...” trigger apply_override: soft constraints are erased; disclosed hard facts are retained (the simulator will not re-send them); discarded soft tokens are blocked from FTS; history before the override is excluded from query construction; asked is reset so other can fire again. This design lifted override Hit@10 from approximately 0.80 to 0.97 in intermediate public-set iterations.

Boundary don't-care. Utterances of the form “I don't have a preference for X” mark attribute X as don't-care so the agent neither filters nor re-asks it.

Audience and family. Gift/relationship phrases (e.g., for my son, for him) set an internal audience label used only in ranking. Product-family patterns disambiguate senses such as dress garment vs dress shoes. Over-broad audience matching on category strings (e.g., treating “Women Dresses” as gift-for-her) was measured to cost approximately 0.01 TechnicalScore and was removed.

B. Hybrid Retrieval

Lexical lane. Products are indexed in an in-process SQLite FTS5 table. Queries combine OR over session terms with AND-oriented constraint tokens. After override, pre-override message text and discarded soft tokens are omitted to avoid resurrecting abandoned goals.

Dense lane (optional). starter/dense.py builds a hashed character n-gram embedding over titles/attributes (NumPy). At query time, DenseIndex supplies a recall set ($K=80$) and a cosine-style score fused into reranking (default weight 4.5 for hash; higher weight for optional MiniLM). Without NumPy the agent degrades to pure FTS. MiniLM (sentence-transformers) remains explicit opt-in via SHOPPILOT_DENSE=minilm and is not required for the reported 0.897 score.

C. Reranking and Precision Control

Candidates are scored with a linear combination of normalized BM25, dense similarity, constraint coverage (disclosed/override phrases preferred over soft), super-linear combo exact-match bonuses when multiple session phrases hit product text, audience/family hit–miss adjustments, and category-tail bonuses when session category terms align with product leaf categories (exact bonus 10.0; partial 2.5).

Early-turn precision. Because the evaluator ends a session at first Top-10 hit, a lucky but poorly ranked early hit freezes a weak reciprocal rank. ShopPilot therefore emits Top-1 only for the first precision turns (default 3, configurable) and can extend precision until a minimum constraint count is reached (capped by turn). This peer-validated lever primarily improves MRR without sacrificing eventual Hit@10.

D. Clarification Policy

The official simulator reveals hidden product constraints only when ask_attribute is set. ShopPilot always pairs recommendations with a question. The first ask is other (simulator catch-all that dumps remaining constraints). Subsequent asks follow a static order color → material → style → ... with limited pool-split reordering among a safe subset. Public-set A/B tests rejected pure maximum information-gain over the candidate pool (TechnicalScore dropped to approximately 0.72 historically) because high-entropy brand/store splits rarely match the simulator’s hidden classifier. Facet tags extracted from the live pool still ground message wording (e.g., offering concrete color options).

A second other ask when evidence remains thin (SHOPPILOT_OTHER_TWICE, default on) is another measured lever for residual uncertainty after the first disclosure dump.

E. Optional LLM Path (Non-Default)

When SHOPPILOT_LLM=1 and a provider key is present, the agent may call a light slot normalizer (always or low-confidence only) and/or rerank the already-retrieved top candidates. Outputs are schema-validated against the official ask enum; failures fall back to rules. The judged default remains offline. Reported token usage on the hash evaluation path is zero.

IV. Experimental Setup

Catalog. Frozen 50,000-product JSONL from the TechJam participant kit (Amazon Reviews 2023 Clothing_Shoes_and_Jewelry derivative). Read-only; SHA256 verification supported.

Evaluation. Official python -m evaluator.local_evaluator on data/public_set.jsonl (N=200). Scenarios: buying (80), browsing (80), intent_override (30), boundary (10). We do not modify evaluator/ or public labels when reporting scores. Private 800-session set was not used.

Implementation. Python 3.10+ (tested 3.13). Core agent approximately 1.6k LOC in starter/agent.py plus dense, rewrite, and optional LLM helpers. Unit tests cover slots, rewrite, dense, and evaluator smoke paths.

Baseline. Organizer weak BM25 starter: Hit@10=0.125, MRR=0.068034, MTTC=9.81, Efficiency=0.119, TechnicalScore=0.10671 (docs/baseline_results.json).

V. Results

Table I reports the primary public-set comparison for the default offline hash configuration (results.json). ShopPilot improves TechnicalScore from 0.107 to 0.897 (approximately 8.4×). Hit Rate@10 rises from 12.5% to 97.0%; MRR from 0.068 to 0.836; MTTC falls from 9.81 to 2.93 turns.

TABLE I. Public 200-session results (official evaluator).

System	Hit@10	MRR	MTTC	Efficiency	TechnicalScore	Tokens
Weak BM25 starter	0.125	0.068	9.81	0.119	0.107	0
ShopPilot (hash, default)	0.970	0.836	2.93	0.807	0.897	0

Table II breaks down ShopPilot by scenario. Buying converts fastest (MTTC 2.54). Intent override remains slower by protocol (MTTC 4.17) yet retains Hit@10 0.967 and strong MRR 0.866 after soft-only wipe and ask reset. Boundary reaches Hit@10 1.000 and MRR 0.933 on N=10.

TABLE II. ShopPilot per-scenario metrics (hash default, N=200).

Scenario	N	Hit@10	MRR	MTTC
Buying	80	0.975	0.833	2.54
Browsing	80	0.963	0.816	2.85
Intent override	30	0.967	0.866	4.17
Boundary	10	1.000	0.933	3.00
Overall	200	0.970	0.836	2.93

Intermediate stack. An earlier public hash run (results_hash.json) scored TechnicalScore 0.789 (Hit 0.93, MRR 0.553, MTTC 3.08). The lift to 0.897 is attributed primarily to precision Top-1 early turns, category-tail exact bonuses, and a second other ask under thin evidence—each gated by environment flags for ablation.

Negative results (selected). Pure max-IG ask ordering over the live pool reduced TechnicalScore (historical approximately 0.72). Aggressive stemming and some RRF fusions hurt held-out style checks in related experiments. Optional MiniLM dense improved paraphrase cases in isolation but was not required for the default 0.897 hash path.

VI. Discussion

Why offline hybrid works here. The simulator's information channel is dominated by structured ask_attribute disclosures and lexical overlap with catalog fields (title, categories, features). Once constraints are captured with provenance, BM25 + coverage scoring recovers the target ASIN for most sessions—consistent with CQ literature that ties good questions to large early-precision gains [1], [8]. Dense hash n-grams help residual paraphrase gaps without a GPU.

MRR as the critical lever. TechnicalScore weights MRR at 0.30. Ending sessions on a Top-10 hit that is not rank-1 caps reciprocal rank. Precision turns deliberately delay broad Top-10 emission until the state is rich enough to place the target higher—trading a small risk of later hit for large MRR gains (0.55 → 0.84 class improvement between intermediate and final stacks).

Ask policy vs. information gain. Theory favors entropy-reducing questions [2]; our public-set ablation shows that naively maximizing pool split entropy is not sufficient when the oracle's attribute channel differs from high-entropy catalog facets. Corpus-grounded facet mentions in messages [5] remain useful even when ask order stays static.

Public-set overfitting risk. All reported numbers use the same 200 sessions available to every team. High public scores can exploit simulator phrasing. We mitigate via unit tests, scenario breakdowns, documented rejected policies, and an offline default that judges can run with network disabled. The private 800 remains the only pristine ranking; we did not access it. Empirical CQ studies also warn that lab/simulator answer behavior can diverge from free users [3].

Positioning vs leaderboard claims. Public GitHub READMEs report TechnicalScores from approximately 0.75 to 0.97. ShopPilot at 0.897 is competitive on the official metric while prioritizing reproducibility, zero tokens, and explicit failure analysis. Final hackathon ranking also includes innovation, impact, feasibility, and presentation beyond TechnicalScore alone.

VII. Limitations and Future Work

(1) Approximately 3% of public sessions still miss Top-10; near-paraphrase feature sentences and sparse titles remain failure modes. (2) Ask order after other is intentionally static relative to this simulator; transferring to a different user model may need adaptive policies. (3) Override MTTC is structurally bounded by disclosure timing. (4) Optional LLM NLU/rerank needs keys and is not the submission default. (5) No private-800 numbers. Future work: calibrated dense/lexical gating, learned rerankers trained only on development folds, robust synonym normalization, and stress tests under paraphrased user wording.

VIII. Reproducibility

Repository: <https://github.com/algorathem/techjam2026-shopping-copilot> (local mirror used for this report: Downloads/techjam2026-shopping-copilot). Entry point: starter/agent.py class Agent. Reproduce:

```
export SHOPPILOT_DENSE=hash; python -m evaluator.local_evaluator
```

Expected aggregate fields in results.json: hit_rate_at_10≈0.97, mrr≈0.836, mttr≈2.93, recommended_technical_score≈0.897, reported_token_usage total_tokens=0. Interactive demo: python cli_chat.py --dense hash. Tests: python -m unittest tests.test_agent_slots tests.test_rewrite tests.test_dense tests.test_evaluator -q.

IX. Ethics and Data

Catalog and sessions derive from Amazon Reviews 2023 (McAuley Lab, UCSD); see DATA_ATTRIBUTION.md. No personal user data beyond synthetic evaluation sessions is processed. No secrets are committed. The agent does not mutate the catalog or inject ASINs.

X. Conclusion

ShopPilot demonstrates that a carefully engineered offline multi-turn retrieve-and-ask agent can approach strong TechnicalScores on the TechJam 2026 shopping copilot benchmark without LLMs or GPUs. The decisive ingredients are provenance-aware dialog state (especially override hygiene), hybrid in-memory retrieval, constraint-coverage ranking, early-turn precision control, and a simulator-aligned clarification policy. On 200 public sessions the default hash system reaches TechnicalScore 0.897 (Hit@10 0.970, MRR 0.836, MTTC 2.93) versus 0.107 for the official weak baseline, at zero token cost.

Appendix A — Environment Flags (Default = Reported Path)

Variable	Default	Role		
SHOPPILOT_DENSE	hash/auto	none	hash	minilm dense backend
SHOPPILOT_PRECISION_TURNS	3	Early turns emit Top-1 only		
SHOPPILOT_OTHER_TWICE	1	Second other ask if thin evidence		
SHOPPILOT_CATEGORY_TAIL	1	Category-tail exact/partial bonus		
SHOPPILOT_LLM	off	Optional network LLM features		
SHOPPILOT_LLM_SLOTS	off	off	lowconf	always slot NLU
SHOPPILOT_LLM_RERANK	off	Optional top-candidate rerank		

Appendix B — Implementation Footprint

Approximate sizes: starter/agent.py 1632 lines; dense.py 235; llm_slots.py 361; llm_rerank.py 197; rewrite.py 64. Supporting docs include DEVPOST.md, DEMO_VIDEO_SCRIPT.md, architecture diagram, miss-category reports, and unit tests under tests/.

References

- [1] M. Aliannejadi, H. Zamani, F. Crestani, and W. B. Croft, “Asking Clarifying Questions in Open-Domain Information-Seeking Conversations,” in Proc. SIGIR, 2019. arXiv:1907.06554.
- [2] F. Mangili, D. Broggini, A. Antonucci, M. Alberti, and L. Cimasoni, “A Bayesian Approach to Conversational Recommendation Systems,” arXiv:2002.05063, 2020.
- [3] J. Zou, E. Kanoulas, and Y. Liu, “An Empirical Study of Clarifying Question-Based Systems,” arXiv:2008.00279, 2020 (parts in CIKM 2020).
- [4] A. Montazeralghaem, G. Tennenholtz, C. Boutilier, and O. Meshi, “Asking Clarifying Questions for Preference Elicitation With Large Language Models,” arXiv:2510.12015, 2025.

- [5] A. M. Krasakis, A. Yates, and E. Kanoulas, “Corpus-informed Retrieval Augmented Generation of Clarifying Questions,” arXiv:2409.18575, 2024.
- [6] A. Rastogi, D. Hakkani-Tür, and L. Heck, “Scalable Multi-Domain Dialogue State Tracking,” arXiv:1712.10224, 2017.
- [8] X. Ren, H. Yin, T. Chen, H. Wang, Z. Huang, and K. Zheng, “Learning to Ask Appropriate Questions in Conversational Recommendation,” in Proc. SIGIR, 2021. arXiv:2105.04774.
- [9] Z. Wang, Y. Tu, C. Rosset, N. Craswell, M. Wu, and Q. Ai, “Zero-shot Clarifying Question Generation for Conversational Search,” arXiv:2301.12660, 2023.
- [10] Y. Luo, H. Fang, and Z. Sun, “Research on Conversational Recommender System Considering Consumer Types,” arXiv:2508.13209, 2025.
- [11] T. Pham and D. Q. Nguyen, “JPIS: A Joint Model for Profile-based Intent Detection and Slot Filling with Slot-to-Intent Attention,” arXiv:2312.08737, 2023.
- [12] F. Cai, W. Zhou, F. Mi, and B. Faltings, “SLIM: Explicit Slot-Intent Mapping with BERT for Joint Multi-Intent Detection and Slot Filling,” arXiv:2108.11711, 2021.
- [13] Emergent Mind, “Dialogue State Tracking (DST),” topic survey. [Online]. Available: <https://www.emergentmind.com/topics/dialogue-state-tracking-dst>
- [14] Emergent Mind, “State-Update Multi-Turn Dialogue Strategy,” topic survey. [Online]. Available: <https://www.emergentmind.com/topics/state-update-multi-turn-dialogue-strategy>
- [16] TechJam 2026, “techjam-conversational-search” participant kit and evaluator. [Online]. Available: <https://github.com/TechJam2026/techjam-conversational-search>

Acknowledgment—Built on the TechJam2026/techjam-conversational-search participant kit and evaluator [16]. Metrics in Section V are taken from the author’s results.json produced by the unmodified official local evaluator. Related-work quotes and arXiv identifiers were curated by the author from public abstracts; full bibliographic venues should be verified against the PDF versions of [1]–[12] for camera-ready use.

Document type: IEEE-style technical report for hackathon documentation (not a peer-reviewed IEEE publication). Generated for TikTok TechJam 2026 Track 4 submission materials.